

```
        // RECURSIVE CASE
        return isOddTailCall(number - 1, !inversionAccum);
    }
}

document.write(isOdd(42) + "<br />");
document.write(isOdd(99) + "<br />");
</script>
```

If this Python and JavaScript code is run by an interpreter that supports tail call optimization, calling `isOddTailCall(100000)` won't result in a stack overflow. However, tail call optimization is still much slower than simply using the `%` modulus operator to determine oddness or evenness.

If you think recursion, with or without tail recursion, is an incredibly inefficient way to determine whether a positive integer is odd, you are absolutely correct. Unlike iterative solutions, recursion can fail from stack overflows. Adding tail call optimization to prevent stack overflows doesn't fix the efficiency flaws of using recursion inappropriately. As a technique, recursion is not automatically better or more sophisticated than iterative solutions. And tail recursion is never a better approach than a loop or other simple solution.